

西安交通大学

课程实验报告

实验名称: 对称、非对称加解密

姓名: 林圣翔

学号: 2223312202

专业班级: 计试 2201

指导教师: 马小博教授

报告日期: 2025.12.14



一、实验目的

1. 加深对非对称，对称加解密算法、散列函数的理解。
2. 了解常用加密工具包以及相关库函数的使用。
3. 了解 ssl 协议。

二、实验平台

ubuntu 虚拟机

三、实验原理

1. 对称加密

采用单钥密码系统的加密方法，同一个密钥可以同时用作信息的加密和解密，这种加密方法称为对称加密，也称为单密钥加密。在对称加密算法中常用的算法有：DES、3DES、TDEA、Blowfish、RC2、RC4、RC5、IDEA、SKIPJACK 等。对称加密算法的缺点是在数据传送前，发送方和接收方必须商定好密钥，然后使双方都能保存好密钥。其次如果一方的密钥被泄露，那么加密信息也就不安全了。另外，每对用户每次使用对称加密算法时，都需要使用其他人不知道的唯一密钥，这会使得收、发双方所拥有的钥匙数量巨大，密钥管理成为双方的负担。

2. 非对称加密

非对称加密算法需要两个密钥：公开密钥（publickey）和私有密钥（privatekey）。公开密钥与私有密钥是一对，如果用公开密钥对数据进行加密，只有用对应的私有密钥才能解密；如果用私有密钥对数据进行加密，那么只有用对应的公开密钥才能解密。因为加密和解密使用的是两个不同的密钥，所以这种算法叫作非对称加密算法。非对称加密算法实现机密信息交换的基本过程是：甲方生成一对密钥并将其中的一把作为公用密钥向其它方公开；得到该公用密钥的乙方使用该密钥对机密信息进行加密后再发送给甲方；甲方再用自己保存的另一把专用密钥对加密后的信息进行解密。

3. 散列函数 Hash

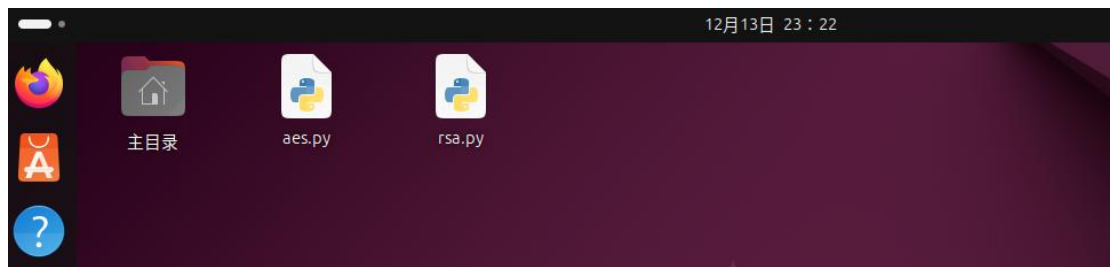
散列函数 Hash，一般翻译做“散列”，也有直接音译为“哈希”的，就是把任意长度的输入（又叫做预映射，pre-image），通过散列算法，变换成固定长度的输出，该输出就是散列值。这种转换是一种压缩映射，也就是，散列值的空间通常远小于输入的空间，不同的输入可能会散列成相同的输出，而不可能从散列值来唯一的确定输入值。简单的说就是一种将任意长度的消息压缩到某一固定长度的消息摘要的函数。

四、实验步骤

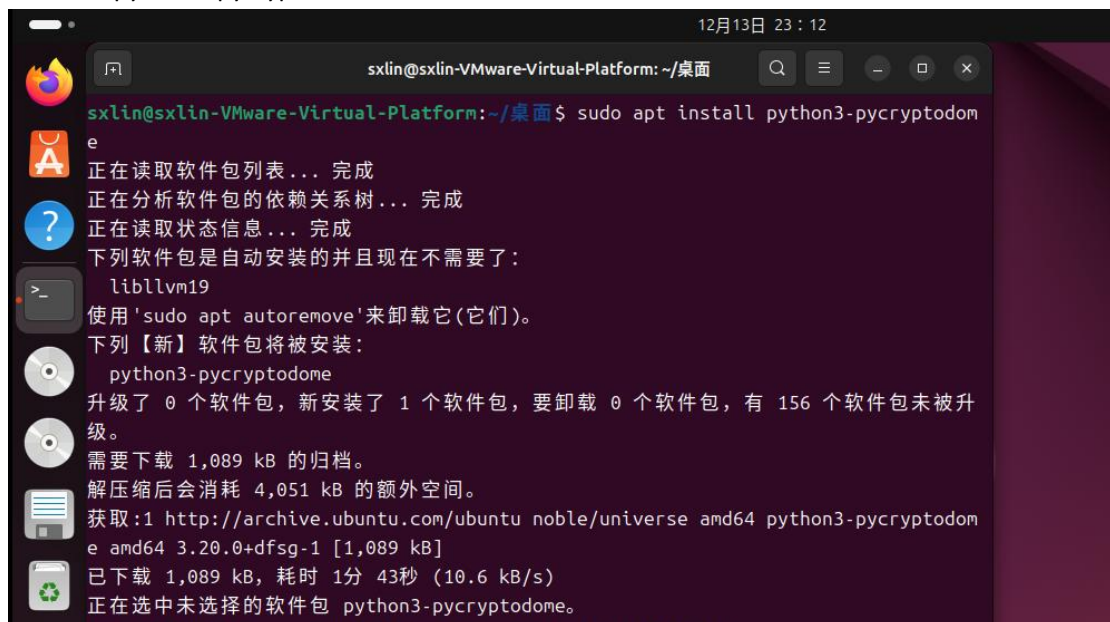
1. 对称加密算法的字符串的加解密

本实验中，需要使用 python 对字符串进行加解密。

1.1.将压缩包中的“实验三 aes.py”和“实验三 rsa.py”（暂时用不到，在“使用 RSA 的加解密”中用到）复制至虚拟机上。



1.2.安装 python3-pycryptodome。

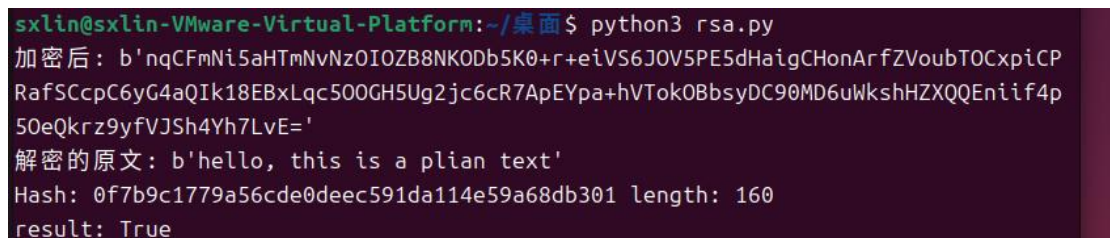


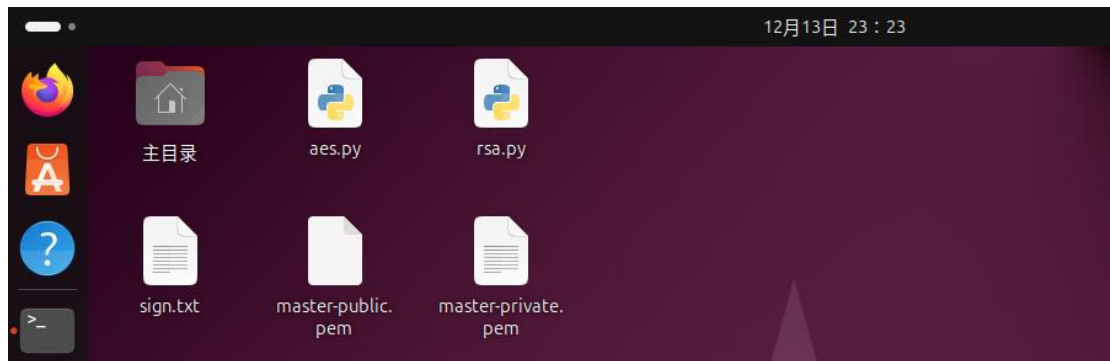
1.3.使用 AES 算法对字符串进行加密。



2.使用 RSA 的加解密

2.1.运行 python3 rsa.py，生成 sign.txt、master-public.pem、master-private.pem 三个文件。





2.2.查看 master-public.pem、master-private.pem 这两个公私钥文件。

```

sxlin@sxlin-VMware-Virtual-Platform: ~/桌面
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ cat master-public.pem
-----BEGIN PUBLIC KEY-----
MIGfMA0GCQsQGSIB3DQEBAQUAA4GNADCBiQKBgQCoD/sc92ycUftnyVieeyYp9WzN
06UUR2TnVZZpK0VyHAVbyiRio+rmGBu376BUC+41R32eMycrr0TEEx1EePCmImrbD
Y8l2NIxz4p2skyHuPCjxXUpBIds8Q413avb7BjbLciShW0bUH08gSuj7enojctLA
oHn8TqngodPpM9/DiQIDAQAB
-----END PUBLIC KEY-----sxlin@sxlin-VMware-Virtual-Platform:~/桌面$
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ cat master-private.pem
-----BEGIN RSA PRIVATE KEY-----
MIICWwIBAAKBgQCoD/sc92ycUftnyVieeyYp9WzN06UUR2TnVZZpK0VyHAVbyiRi
o+rmGBu376BUC+41R32eMycrr0TEEx1EePCmImrbDY8l2NIxz4p2skyHuPCjxXUpB
Ids8Q413avb7BjbLciShW0bUH08gSuj7enojctLAoHn8TqngodPpM9/DiQIDAQAB
AoGAAGFCzSXUfrQ/AWN2XQRPax05oGZPI1LLW7PP2gxWmpW5U07WNUK/di5cph3
ZgUws85ZB3Xzwy0m7wleWd583laLCurdhLFi5Pv4r/KHByVC8pQs4pH10DUP0dYT
iZfcedj43GYLj0Hb5l2jkk++BPo0a+qQ2egUB0IxzffUBIECQDMCm8D0DVGikis
Hs2ZsLeqDBxT3UAakyODAJPpu3E3roUeWxb0tqL0kvA0FLDpXj6WYm/vguKyNi3Y
U1XAmRWhAkeA0twX80BS8k4Ltr1UIEBexLjqD8SfLTl02b9qDz2ccv3Pbj3EuJC0
FfniM43B4X7DKjzqswGITrUTQMb798qU6QJAPSpLiQ2KevDtRBufyp9Fg63VzSW
COFe3eEFixqvn9+DLExmE+WR72oA87xU2QBvhsNPht8XkhDHwXeJ10iMwQJAbY2Q
msPSFLZb+6v0g5st7JMBuQONOC/o0dXrwtNd75jTJxHMnaABENHtp9mNkRoHg/c
WPjY2xvTviTfSCx0QJAG2lh69WqUz8newVcygAylMUHKxw/VGIknLf3aJqyYnDK
QtDD0ZGiVsh3G9rYcPHMtnCbyeodqqnZrN8mIW0hw==
-----END RSA PRIVATE KEY-----sxlin@sxlin-VMware-Virtual-Platform:~/桌面$

```

2.3.使用公钥加密，并使用私钥解密：需要确保这些代码是在公私钥已生成的环境下运行。

```

sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ python3 rsa.py
加密后: b'SQuvk+woaSJhPNumFV96rVKyCIMLOKQcT1MTP7ebc7ThBa8GI/W2LnCKbjI6dMGx4TUtHq
lq3upttKW9LnneX01rZkcJwTQ6Jhe7UqjFG22FhF2Pjt50KADQvM0IVew8dy98yh/8noSxtaUL3tk3D1
eZXyqPy1AUAIL7gQmdN0A='
解密的原文: b'hello, this is a plian text'
Hash: 0f7b9c1779a56cde0deec591da114e59a68db301 length: 160
result: True

```

3.数字签名

签名验证结果

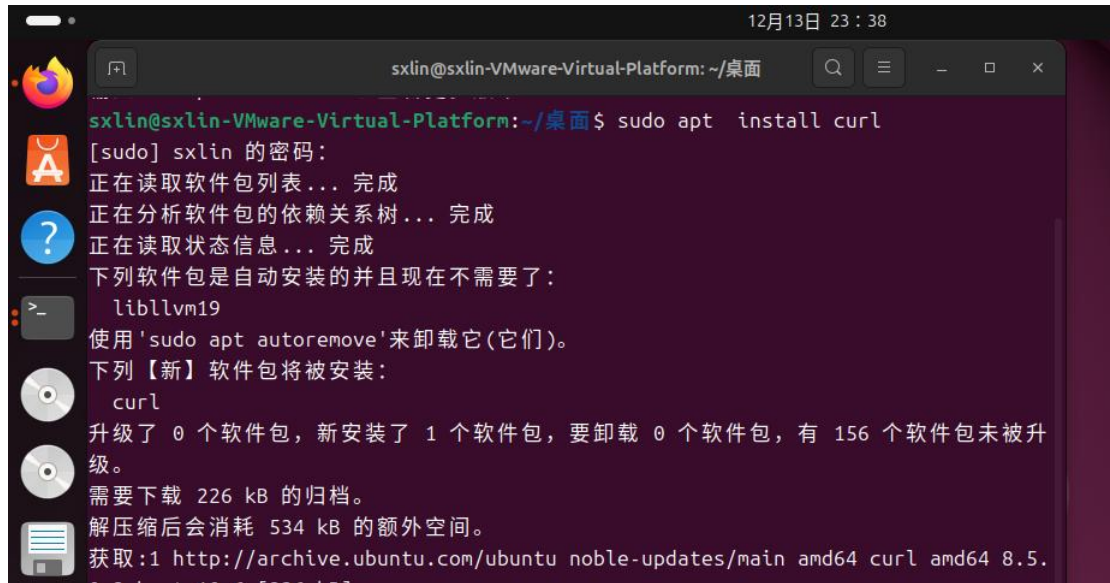
```

Hash: 0f7b9c1779a56cde0deec591da114e59a68db301 length: 160
result: True

```

4.抓包观察 ssl 协议通信握手过程

4.1.安装 curl。



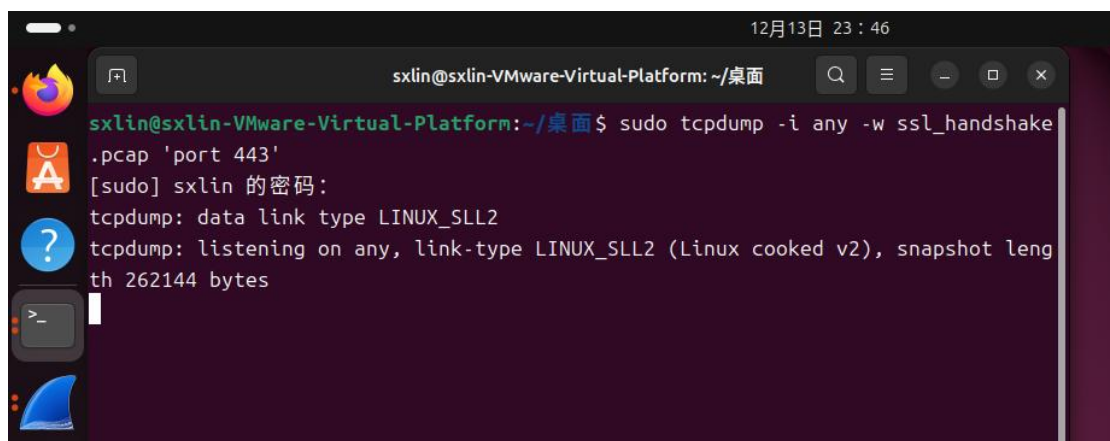
12月13日 23:38

```

sxlin@sxlin-VMware-Virtual-Platform: ~/桌面
[sudo] sxlin 的密码:
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了:
  libllvm19
使用 'sudo apt autoremove'来卸载它(它们)。
下列【新】软件包将被安装:
  curl
升级了 0 个软件包, 新安装了 1 个软件包, 要卸载 0 个软件包, 有 156 个软件包未被升级。
需要下载 226 kB 的归档。
解压缩后会消耗 534 kB 的额外空间。
获取:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.

```

4.2.运行 `sudo tcpdump -i any -w ~/桌面/ssl_handshake.pcap 'tcp port 443'`, 只抓取目标或源端口是 443(HTTPS 默认端口)的 TCP 流量,并把抓到的原始网络数据保存到 `ssl_handshake.pcap` 文件中。



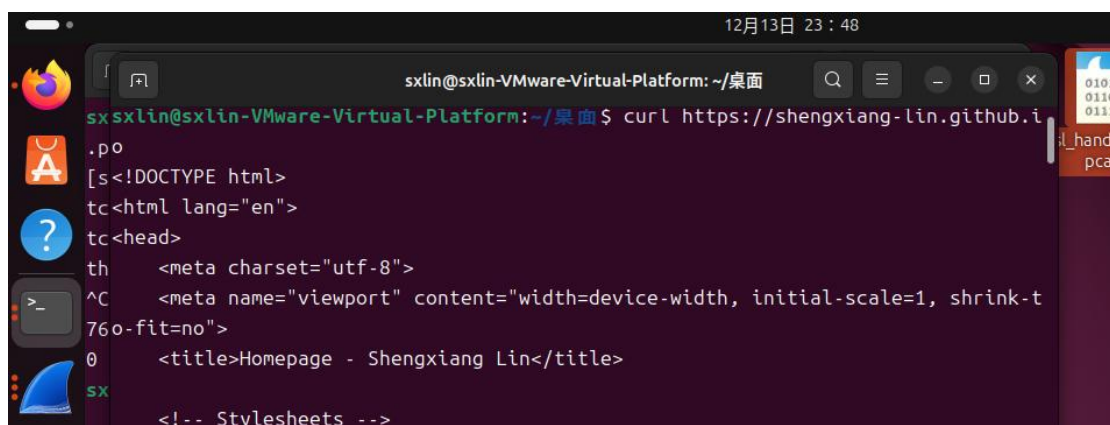
12月13日 23:46

```

sxlin@sxlin-VMware-Virtual-Platform: ~/桌面
[sudo] sxlin 的密码:
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes

```

4.3.打开另一个终端, 使用 `curl` 访问 `https://shengxiang-lin.github.io` 网站,触发握手。



12月13日 23:48

```

sxlin@sxlin-VMware-Virtual-Platform: ~/桌面
[sudo] sxlin 的密码:
curl https://shengxiang-lin.github.io
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
  <title>Homepage - Shengxiang Lin</title>
  <!-- Stylesheets -->

```

4.4.终止 `sudo tcpdump -i any -w ~/桌面/ssl_handshake.pcap 'tcp port 443'`的运行。

```
12月13日 23:48
sxlin@sxlin-VMware-Virtual-Platform: ~/桌面
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ sudo tcpdump -i any -w ssl_handshake
.pcap 'port 443'
[sudo] sxlin 的密码:
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot leng
th 262144 bytes
^C765 packets captured
765 packets received by filter
0 packets dropped by kernel
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$
```

4.5. 打开 ssl_handshake.pcap。

由于.github.io 等较新的服务域名普遍支持更现代的 TLS 1.3 协议，其握手过程被极大地简化。因此，在抓包分析时，通常只能观察到明文的 Client Hello 和 Server Hello 两个初始阶段，后续的证书交换、密钥验证等步骤均被合并并进行了加密传输。

Client Hello

12月14日 00:04

ssl_handshake.pcap

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

应用显示过滤器... <Ctrl-/>

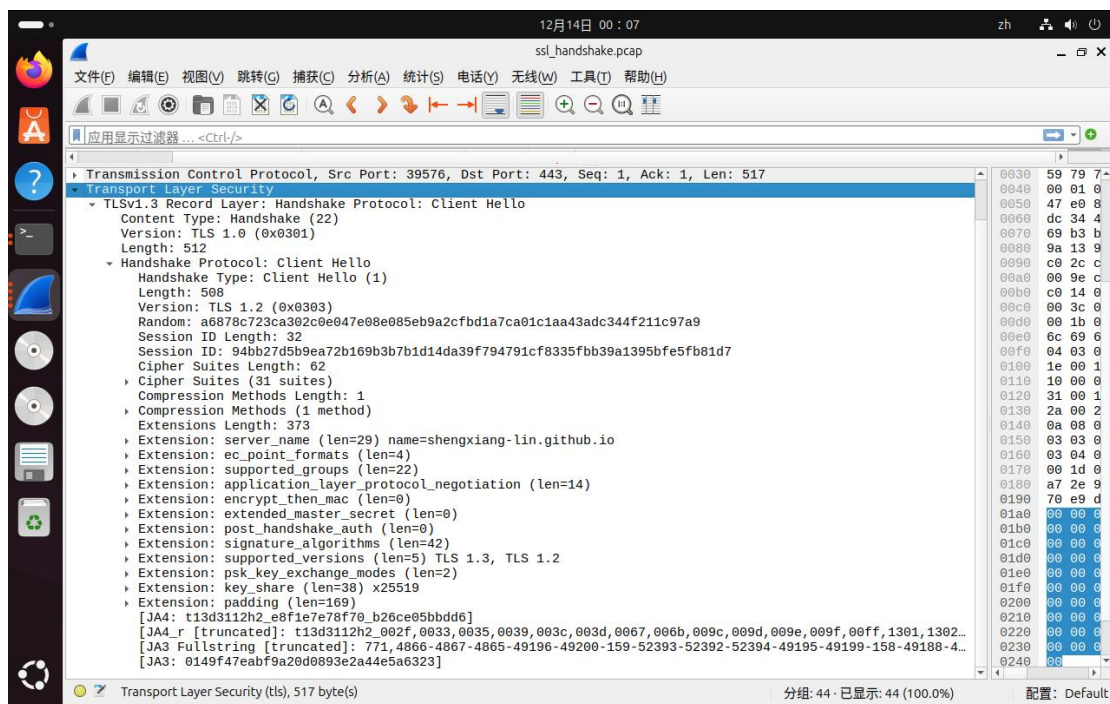
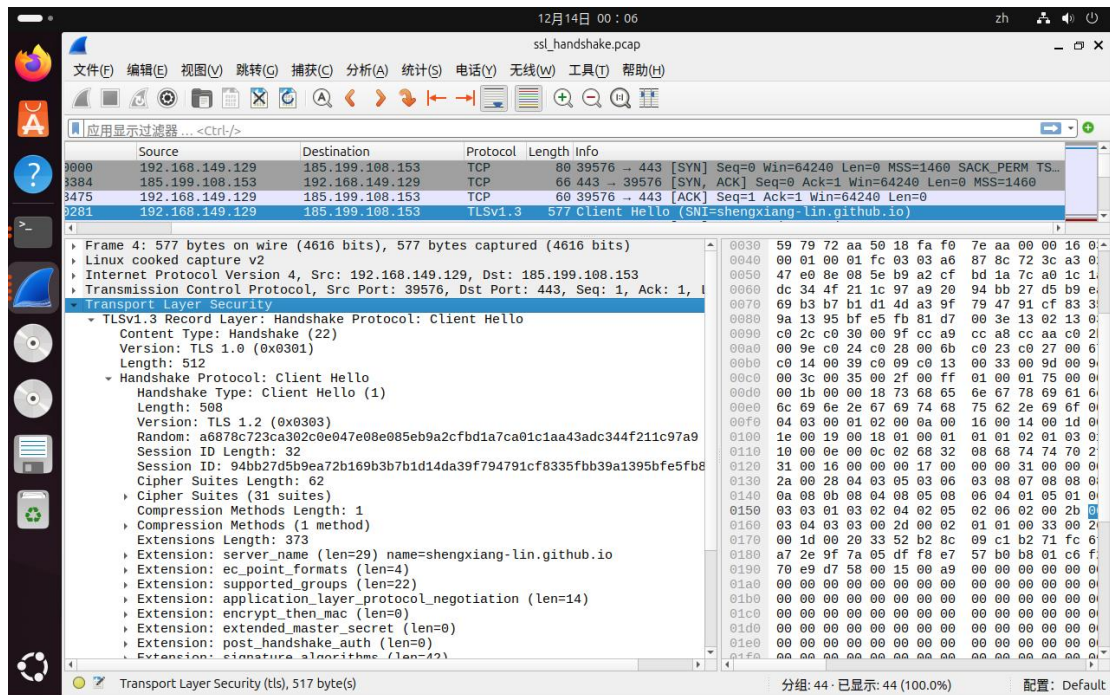
No.	Source	Destination	Protocol	Length	Info
3000	192.168.149.129	185.199.108.153	TCP	80	39576 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TS...
3084	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
3475	192.168.149.129	185.199.108.153	TCP	60	39576 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
3501	192.168.149.129	185.199.108.153	TLSv1.3	577	Client Hello (SNI=shengxiang.lin.github.io)
3440	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [ACK] Seq=1 Ack=518 Win=64240 Len=0
3449	185.199.108.153	192.168.149.129	TLSv1.3	1472	Server Hello, Change Cipher Spec, Application Data
3929	192.168.149.129	185.199.108.153	TCP	60	39576 → 443 [ACK] Seq=518 Ack=1413 Win=65535 Len=0
3969	185.199.108.153	192.168.149.129	TLSv1.3	4071	Application Data, Application Data, Application Data, Applica...
4050	192.168.149.129	185.199.108.153	TCP	60	39576 → 443 [ACK] Seq=518 Ack=5424 Win=65535 Len=0
3528	192.168.149.129	185.199.108.153	TLSv1.3	124	Change Cipher Spec, Application Data
3267	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [ACK] Seq=5424 Ack=582 Win=64240 Len=0
3677	192.168.149.129	185.199.108.153	TLSv1.3	146	Application Data
3362	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [ACK] Seq=5424 Ack=668 Win=64240 Len=0
3613	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [ACK] Seq=5424 Ack=736 Win=64240 Len=0
4034	185.199.108.153	192.168.149.129	TLSv1.3	125	Application Data
4334	192.168.149.129	185.199.108.153	TLSv1.3	91	Application Data
4959	185.199.108.153	192.168.149.129	TCP	66	443 → 39576 [ACK] Seq=5489 Ack=767 Win=64240 Len=0
3210	185.199.108.153	192.168.149.129	TLSv1.3	8938	Application Data, Application Data, Application Data, Applica...
3294	192.168.149.129	185.199.108.153	TCP	60	39576 → 443 [ACK] Seq=767 Ack=14367 Win=65535 Len=0
4798	185.199.108.153	192.168.149.129	TLSv1.3	2884	Application Data
4895	192.168.149.129	185.199.108.153	TCP	60	39576 → 443 [ACK] Seq=767 Ack=17191 Win=65535 Len=0

Frame 4: 577 bytes on wire (4616 bits), 577 bytes captured (4616 bits)

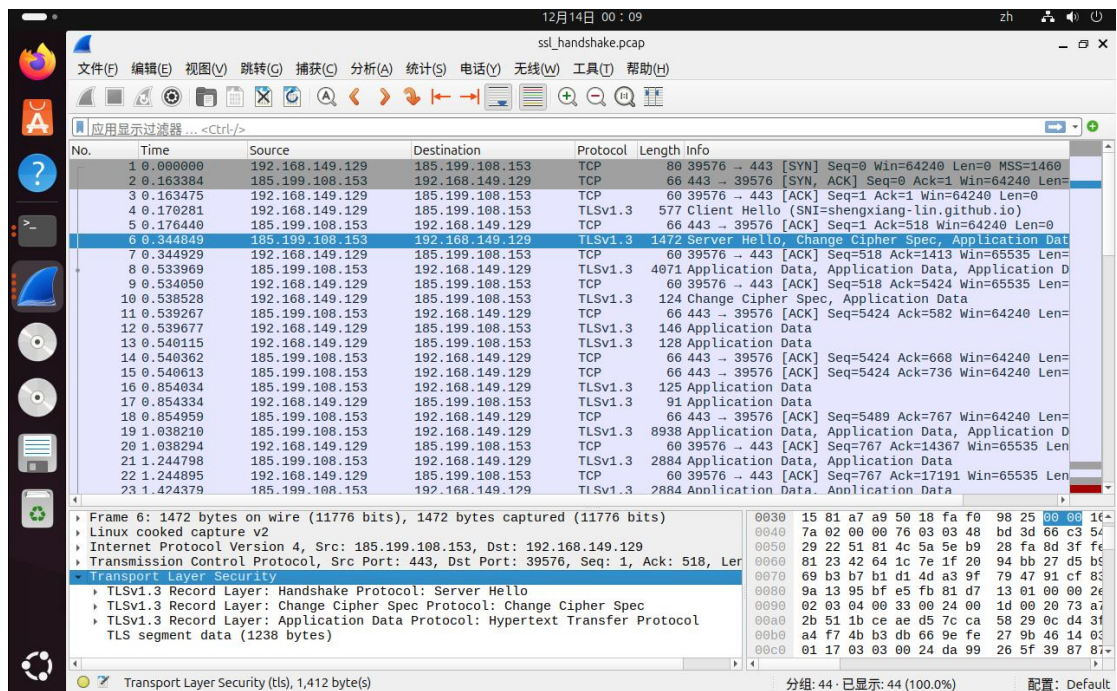
- Linux cooked capture v2
- Internet Protocol Version 4, Src: 192.168.149.129, Dst: 185.199.108.153
- Transmission Control Protocol, Src Port: 39576, Dst Port: 443, Seq: 1, Ack: 1, Len: 577
- Transport Layer Security

ssl_handshake.pcap

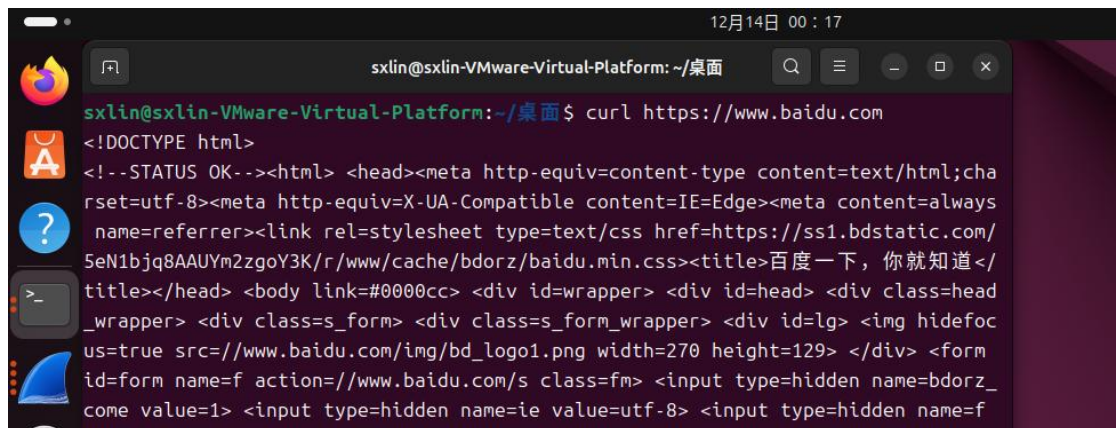
分组: 44 · 已显示: 44 (100.0%) 配置: Default



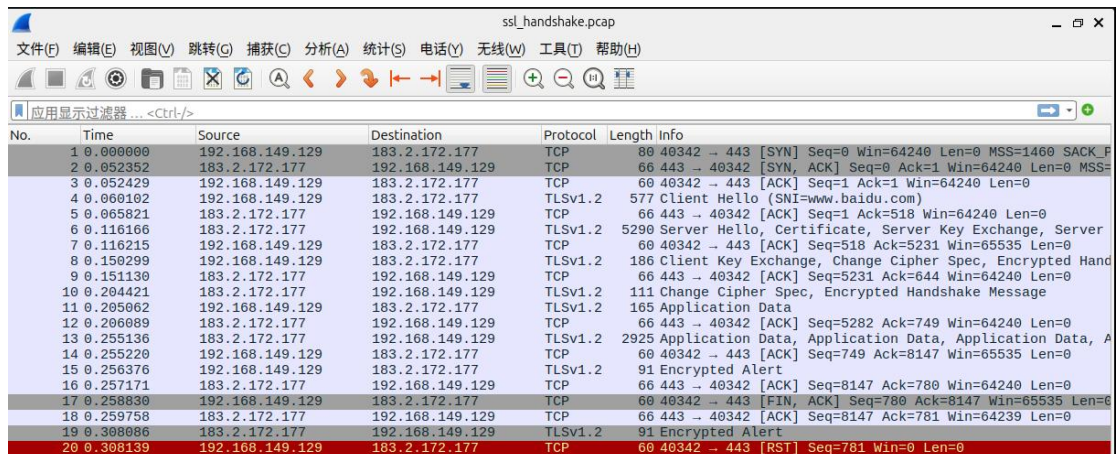
Server Hello



4.6.参考 4.2-4.4 的步骤，使用 curl 访问 <https://www.baidu.com> 网站,触发握手。



对于支持 TLS 1.2 协议的 .com 等传统域名，其握手过程遵循经典的四步模式，在抓包中清晰可见，包括：Client Hello、Server Hello（与证书等合并）、Client Key Exchange 及最终的确认。



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_F
2	0.052352	183.2.172.177	192.168.149.129	TCP	60	443 → 40342 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=
3	0.052429	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [ACK] Seq=1 Ack=1 Win=64240 Len=0
4	0.060102	192.168.149.129	183.2.172.177	TLSv1.2	577	Client Hello (SNI=www.baidu.com)
5	0.065821	183.2.172.177	192.168.149.129	TCP	66	443 → 40342 [ACK] Seq=1 Ack=518 Win=64240 Len=0
6	0.116166	183.2.172.177	192.168.149.129	TLSv1.2	5290	Server Hello, Certificate, Server Key Exchange, Server
7	0.116215	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [ACK] Seq=518 Ack=5231 Win=65535 Len=0
8	0.150299	192.168.149.129	183.2.172.177	TLSv1.2	186	Client Key Exchange, Change Cipher Spec, Encrypted Hand
9	0.151130	183.2.172.177	192.168.149.129	TCP	66	443 → 40342 [ACK] Seq=5231 Ack=644 Win=64240 Len=0
10	0.204421	183.2.172.177	192.168.149.129	TLSv1.2	111	Change Cipher Spec, Encrypted Handshake Message
11	0.205062	192.168.149.129	183.2.172.177	TLSv1.2	165	Application Data
12	0.206089	183.2.172.177	192.168.149.129	TCP	66	443 → 40342 [ACK] Seq=5282 Ack=749 Win=64240 Len=0
13	0.255136	183.2.172.177	192.168.149.129	TLSv1.2	2928	Application Data, Application Data, Application Data, A
14	0.255220	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [ACK] Seq=749 Ack=8147 Win=65535 Len=0
15	0.256376	192.168.149.129	183.2.172.177	TLSv1.2	91	Encrypted Alert
16	0.257171	183.2.172.177	192.168.149.129	TCP	66	443 → 40342 [ACK] Seq=8147 Ack=780 Win=64240 Len=0
17	0.258830	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [FIN, ACK] Seq=780 Ack=8147 Win=65535 Len=0
18	0.259758	183.2.172.177	192.168.149.129	TCP	66	443 → 40342 [ACK] Seq=8147 Ack=781 Win=64239 Len=0
19	0.308086	183.2.172.177	192.168.149.129	TLSv1.2	91	Encrypted Alert
20	0.308139	192.168.149.129	183.2.172.177	TCP	60	40342 → 443 [RST] Seq=781 Win=0 Len=0

五、思考题

1.回答 AES 加密中 iv 变量的作用。

在 AES 加密模式中，初始化向量 (IV) 的核心作用是引入随机性，确保相同明文与密钥多次加密后生成完全不同的密文。其工作原理是：CBC 模式先将第一个明文块与 IV 进行异或，再加密；后续每个明文块都会先与前一个密文块异或后再加密，形成链式依赖。这一机制能有效掩盖明文模式，提升安全性。但若将 IV 固定（如代码中的 `b'qqqqqqqqqqqqqqqq'`），则随机性丧失，该安全特性将不复存在。实际应用中，需要为每次加密随机生成唯一 IV，并与密文一起传输（IV 无需保密）；解密时使用同一 IV 即可还原明文。

2.思考问题：RSA【公钥加密，私钥解密】和【私钥加密，公钥解密】算法一样吗？为什么？

在数学运算层面，RSA 的【公钥加密，私钥解密】与【私钥加密，公钥解密】所使用的核心算法是相同的，都是模幂运算：加密或签名时做一次幂运算，解密或验证时再做一次幂运算。然而，这两种操作在密码学的目的和安全含义上完全不同，因此在实际中被视为两种独立的应用，不可互换。

【公钥加密，私钥解密】目的是为了保密性。发送者使用接收者公开的公钥加密信息，确保只有拥有对应私钥的接收者才能解密读取，从而实现信息的安全传输。【私钥加密，公钥解密】目的是为了认证性和完整性。发送者使用自己的私钥对信息的摘要进行签名（即“加密”），接收者使用发送者的公钥验证（即“解密”）。这能证明信息确实来自声称的发送者（认证），且在传输中未被篡改（完整性）。

两对密钥的性质截然不同：公钥是公开分发的，私钥必须绝对保密。如果错误地用私钥去加密需要保密的信息，那么任何拿到公钥的人都能解密，这就完全失去了保密的意义。因此，虽然底层数学运算对称，但 RSA 的这两种应用模式服务于根本不同的安全目标。

3.书写数字签名的注释，每行都干了些什么？并任意举一个例子使得 `result=False`。

```
n = b'This is a test message'          # 准备待签名的原始消息
h = SHA.new()                          # 创建 SHA 哈希对象
h.update(n)                            # 将消息更新到哈希对象中，计算消息摘要
print('Hash:',h.hexdigest(),'length:',len(h.hexdigest()*4) # 打印消息摘要值及其位长
sign_txt = 'sign.txt'                  # 定义签名保存文件名
# 使用私钥对消息摘要进行签名
with open('master-private.pem') as f:  # 打开私钥文件
```

```

key = f.read()                # 读取私钥内容
private_key = RSA.importKey(key) # 将 PEM 格式私钥导入为 RSA 密钥对象
hash_obj = SHA.new(n)         # 重新创建消息摘要（必须与验证时一致）
signer = Signature_pkcs1_v1_5.new(private_key) # 创建 PKCS#1 v1.5 签名器
d = base64.b64encode(signer.sign(hash_obj))    # 对消息摘要签名并 Base64 编码
# 将签名保存到文件
f = open(sign_txt,'wb')        # 以二进制写模式打开签名文件
f.write(d)                     # 将编码后的签名写入文件
f.close()                      # 关闭文件
# 使用私钥验证数字签名
with open('master-private.pem') as f: # 打开打开私钥文件
    key = f.read()                # 读取密钥内容
    public_key = RSA.importKey(key) # 导入密钥
    sign_file = open(sign_txt,'r') # 打开存储签名的文件
    sign = base64.b64decode(sign_file.read()) # Base64 解码读取的签名
    h = SHA.new(n)                # 重新计算原始消息的摘要
    verifier = Signature_pkcs1_v1_5.new(public_key) # 创建验证器（本应使用公钥）
    print('result:', verifier.verify(h,sign)) # 验证签名，返回 True 或 False

```

要使 **result: False**，可以修改验证时计算摘要的消息，例如：在验证部分修改这一行，使用不同的消息。

```
h = SHA.new(b'This is a DIFFERENT message')
```

这样修改后，由于验证时计算的消息摘要与签名时计算的消息摘要不同（尽管签名文件仍是原始消息的签名），验证器会发现签名与当前消息不匹配，从而返回 **False**。

4. 详细分析 ssl 协议通信握手过程。

①客户端发起“Client Hello”消息。客户端向服务器发送支持的 TLS 版本、客户端随机数、会话 ID（可选）以及一个按优先级排列的密码套件列表（包含加密算法、密钥交换算法和消息认证码算法等）。

②服务器回应“Server Hello”消息。服务器从客户端提供的列表中选出双方均支持的 TLS 版本和密码套件，同时生成服务器随机数并发送给客户端。服务器可能发送“Server Certificate”消息来传递其数字证书链，以便客户端验证其身份。如果使用基于公钥的密钥交换（如 RSA 或 ECDHE），服务器还会发送“Server Key Exchange”消息（如适用），而“Certificate Request”消息则在需要客户端认证时发出。最后，服务器发送“Server Hello Done”消息表示回应完毕。

③客户端进行验证与密钥交换。客户端验证服务器证书的合法性。随后，客户端根据协商的密钥交换算法生成预主密钥（Pre-master Secret）：若使用 RSA，则用服务器证书中的公钥加密后通过“Client Key Exchange”消息发送；若使用 ECDHE 等算法，则客户端也会生成并发送自己的临时公钥参数。如果需要，客户端会发送自己的“Client Certificate”和“Certificate Verify”消息以完成客户端认证。

④双方生成主密钥并切换至加密通信。此时，客户端和服务器利用客户端随机数、服务器随机数和预主密钥，通过相同的伪随机函数独立计算出一致的主密钥（Master Secret）。主密钥进而被扩展为会话所需的多个密钥块，包括对称加密密钥和消息认证码（MAC）密钥。客户端发送“Change Cipher Spec”通知，表示后续消息将使用协商的密钥进行加密保护，并紧接着发送一个“Finished”消息（包含加密的完整性验证数据）。服务器同样发送“Change Cipher Spec”和“Finished”消息。

⑤握手完成并开始安全数据传输。服务器“Finished”消息验证通过后，双方握手正式结束。随后，双方将使用会话密钥对所有应用层数据进行加密和完整性保护，建立安全的通信信道。

六、实验问题以及解决过程

1. 尝试使用 `pip3 install pycrypto` 安装时，因 Ubuntu 系统默认的“外部管理环境”限制而被阻止。随后改为通过系统包管理器执行 `apt install pycrypto` 出现相同问题。后改为安装 `pycryptodome` 库作为替代。在运行代码时仍出现 `ModuleNotFoundError: No module named 'Crypto'` 的报错。

```
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ python3 aes.py
Traceback (most recent call last):
  File "/home/sxlin/桌面/aes.py", line 7, in <module>
    from Crypto.Cipher import AES
ModuleNotFoundError: No module named 'Crypto'
```

```
sxlin@sxlin-VMware-Virtual-Platform:~/桌面$ python3 rsa.py
Traceback (most recent call last):
  File "/home/sxlin/桌面/rsa.py", line 8, in <module>
    from Crypto import Random
ModuleNotFoundError: No module named 'Crypto'
```

同步将代码中所有 `Crypto` 的引用修改为 `Cryptodome`，最终顺利解决了依赖与导入问题。

2. 在“抓包分析 SSL/TLS 握手过程”中，首先对 <https://shengxiang-lin.github.io> 进行抓包。由于该域名采用较新的 TLS 1.3 协议，其握手流程被大幅简化，因此在抓包结果中只能观察到明文的 Client Hello 与 Server Hello 两个初始阶段。随后，将目标改为 <https://www.baidu.com>（使用 TLS 1.2 协议），成功捕获到了包括证书交换、密钥协商等在内的完整四次握手过程，从而清晰对比了不同 TLS 版本在握手流程上的差异。

七、实验小结

通过本次实验，我系统实践了对称与非对称加密的核心技术。在 AES 加密中，我理解了初始化向量（IV）对增强安全性的关键作用；在 RSA 部分，我通过代码实操理解了公钥加密与数字签名的相同点和区别。实验过程中，我解决了从库安装、模块导入到协议分析的一系列实际问题，例如适配 `pycryptodome` 库，以及通过访问不同网站，直观观察到 TLS 1.3 与 TLS 1.2 在握手流程上的差异。这次实践让我深刻体会到，安全通信是多种密码学技术精密协作的结果，其复杂性与严谨性远超理论认知，增强了我对构建网络信任体系实际挑战的理解。